

AxiomGraph Compute Efficiency: A Self-Optimizing Cognitive Routing Layer for Local GPU and Transformer-Based AI Agents

Abstract

This paper extends the AxiomGraph developmental agent framework into a compute-efficiency architecture. The central claim is that AI-agent systems can become cheaper, faster, and more reliable by learning which cognitive pathways deserve compute and which should be pruned, cached, compressed, or escalated. AxiomGraph does not directly alter transformer kernels or the internal matrix operations of neural networks. Instead, it improves efficiency at the agent orchestration layer by modeling cognition as a dynamic graph of nodes and synapses, where each pathway tracks task success, cost, latency, token usage, GPU memory pressure, risk, and reliability.

The resulting system can operate across local GPUs, cloud transformer APIs, or hybrid local/cloud deployments. It can choose when to use a small local model, a larger model, a cached workflow, a retrieval path, a specialized tool, or a high-assurance evaluator. The key invariant is to maximize successful task completion per unit compute. This makes AxiomGraph both a developmental AI framework and a practical compute optimizer for production AI agents.

1. Introduction

Modern AI agents often waste compute. A simple task may be routed through an unnecessarily large model. A repeated task may be solved from scratch. A workflow may retrieve too much memory, call too many tools, or loop through failed reasoning paths. These inefficiencies increase API costs, GPU usage, latency, and operational complexity.

AxiomGraph reframes this problem. Instead of treating an agent as a static chain of prompts and tools, it treats the agent as an evolving cognitive graph. Nodes represent models, tools, memory systems, evaluators, planners, routers, and specialized modules. Edges represent pathways with measurable properties, including routing strength, trust, success rate, token cost, latency, risk, and plasticity.

This graph allows the agent to learn which cognitive routes work for which task types. Over time, efficient successful routes strengthen, while expensive unreliable routes weaken or are pruned.

2. Core Thesis

The central thesis is:

> AI-agent compute efficiency can be improved by continuously learning the cheapest reliable cognitive route for each task.

This is not a claim that AxiomGraph makes transformer attention itself physically cheaper. Rather, it reduces the frequency, size, and complexity of expensive transformer calls by optimizing the orchestration layer around the model.

3. Compute Efficiency Invariant

The primary invariant is:

`maximize successful task completion per unit compute`

A practical metric is:

`Cognitive Efficiency = Task Success / Compute Cost`

Compute Cost may include:

`token cost + API cost + latency + GPU memory + tool calls + evaluator calls + failed retries`

A route is better when it delivers equal or higher task success at lower total cost.

4. Sources of Agent Compute Waste

Common sources of waste include:

1. Calling large models for easy tasks.
2. Repeating reasoning that has already been solved.
3. Sending too much context to the model.
4. Retrieving irrelevant memory.
5. Overusing tools.
6. Running evaluators for low-risk tasks.
7. Failing to learn which routes worked in the past.
8. Keeping obsolete modules alive.
9. Retrying failed workflows without changing strategy.
10. Using cloud calls when a local model is sufficient.

AxiomGraph addresses these failures through adaptive routing, evaluation, caching, and pruning.

5. Architecture

The compute-efficiency layer contains seven components:

1. Task Profiler
2. Compute Profile Registry
3. Cognitive Router
4. Cache and Reuse Layer
5. Escalation Policy
6. Evaluation Engine
7. Efficiency Ledger

5.1 Task Profiler

The Task Profiler estimates task type, difficulty, risk, token requirements, and required quality.

5.2 Compute Profile Registry

The registry describes available resources:

- local small models;
- local quantized models;
- larger local GPU models;
- cloud transformer APIs;
- embedding models;
- tool calls;
- cached workflows;
- rule-based functions;
- evaluator models.

Each profile records estimated quality, cost, latency, GPU memory, context length, and local/cloud status.

5.3 Cognitive Router

The router chooses the cheapest reliable path. It may select a cache, a simple tool, a local model, or a stronger cloud model depending on the task.

5.4 Cache and Reuse Layer

If a prior answer or workflow has high confidence and low risk, the system can reuse it instead of calling a large model.

5.5 Escalation Policy

The system starts with the lowest-cost acceptable path and escalates only when confidence, risk, or task difficulty requires it.

5.6 Evaluation Engine

After a task is completed, the result is scored for success, accuracy, safety, cost, and user value.

5.7 Efficiency Ledger

Every decision is logged so the system can learn which routes produced the best task-success-per-compute ratio.

6. Local GPU Optimization

For local systems, AxiomGraph can reduce GPU load by choosing among multiple local execution options.

Example routing:

```
classification -> small local classifier
summarization -> quantized local model
coding -> code-specialized local model
complex reasoning -> larger local model
high-risk verification -> evaluator or cloud fallback
```

The system can also learn when to unload unused models, batch similar tasks, limit context length, or use retrieval instead of generation.

7. Cloud Transformer Optimization

For cloud transformer APIs, AxiomGraph can reduce cost by:

- using cheaper models for easy tasks;
- sending only relevant memory;
- compressing context;
- reducing repeated tool loops;
- escalating only when confidence is low;
- separating drafting from verification;
- caching proven workflows.

This can reduce input tokens, output tokens, tool calls, evaluator calls, and failed retries.

8. Hybrid Local/Cloud Operation

The strongest architecture is hybrid:

```
local classifier -> local small model -> local memory/retrieval -> cloud model only when needed -> evaluator only
```

AxiomGraph learns which tasks can remain local and which require stronger remote inference.

9. Digital Synapse Cost Model

Each edge in the cognitive graph can include:

```
{
  "weight": 0.74,
  "trust_score": 0.91,
  "success_rate": 0.82,
  "token_cost": 430,
  "latency_ms": 240,
  "risk_score": 0.18,
  "plasticity": 0.31
}
```

The route score can be estimated as:

```
route_score =
(weight * trust_score * success_rate * task_relevance)
/
(token_cost + latency_cost + risk_cost)
```

This makes cognitive routing explicitly compute-aware.

10. Growth and Pruning for Efficiency

AxiomGraph growth adds specialized modules when they repeatedly improve task success. AxiomGraph pruning removes or weakens pathways that are costly, unreliable, redundant, or unsafe.

Growth improves specialization. Pruning improves efficiency.

Together they create a self-optimizing agent architecture.

11. Product Implication

The compute-efficiency layer may be the strongest near-term product path for AxiomGraph.

The market-facing claim is:

> AxiomGraph reduces AI-agent compute waste by learning the cheapest reliable cognitive route for every task.

This product can be sold before any physical NeuroGenesis hardware exists.

12. Roadmap

Phase 1: Add compute profiles and task profiles.

Phase 2: Track model cost, token cost, latency, GPU memory, and success rate.

Phase 3: Add cache-first and small-model-first routing.

Phase 4: Add confidence-based escalation.

Phase 5: Add pruning of inefficient pathways.

Phase 6: Add dashboards showing cost savings and cognitive efficiency.

Phase 7: Use this digital efficiency architecture as the control layer for future physical NeuroGenesis systems.

13. Conclusion

AxiomGraph can increase compute efficiency for AI agents running locally on GPUs, through transformer APIs, or across hybrid stacks. Its contribution is not a lower-level accelerator kernel, but a higher-level cognitive routing architecture. By tracking which pathways work, what they cost, and when they should be reused or pruned, AxiomGraph can make agents cheaper, faster, more specialized, and more reliable.

The final thesis is:

> Agents should not spend maximum compute on every thought. They should learn the cheapest reliable pathway to success.